CAN Bus Communication System Implementation Report

STM32F4 + Arduino Integration

Industrial Computing & Embedded Systems

Mini-Project Report

Author:

AIT CHADI Lydia
BOUDJEMA Ziad
KOUDOU Chris-Ibrahim
BELKHEIYALAT Mimoun

Professor:

L. NEHAOUA

Submitted: January 3, 2026

Academic Year: 2025–2026

Department of Electrical, Electronic and Automation Engineering
University of Évry Paris-Saclay

Contents

Chapter 1

Introduction

1.1 Project Context

The Controller Area Network (CAN) bus is a robust vehicle bus standard designed to allow microcontrollers and devices to communicate with each other in applications without a host computer. This project implements a CAN bus communication system between an Arduino with a CAN shield and an STM32F4 Discovery board, demonstrating industrial-grade communication protocols in embedded systems.

1.2 Industrial Problematic

Modern industrial and automotive systems require reliable, real-time communication between multiple electronic control units (ECUs). The CAN protocol provides:

- High reliability in noisy environments
- Deterministic message prioritization
- Fault confinement capabilities
- Multi-master bus architecture

1.3 Technical Objectives

- Configure STM32F4 microcontroller for CAN reception
- Implement Arduino-based CAN frame generation
- Develop UART communication bridge to PC
- Create robust error handling and data processing
- Real-time monitoring and display of CAN traffic

1.4 Technical Specifications used in this project

Parameter	**Specification**
CAN Baud Rate	500 kbps
Microcontroller	STM32F407VGT6
CAN Controller	Integrated bxCAN
UART Interface	USART2 via FTDI
PC Interface	USB Serial
Data Format	CAN 2.0B Standard/Extended

Table 1.1: Technical Specifications

Chapter 2

CAN Bus Protocol Overview

2.1 Historical Background and Applications

The CAN bus was originally developed by Bosch in 1986 for automotive applications. Today it's widely used in:

- Automotive systems (engine control, ABS, airbags)
- Industrial automation (PLC networks)
- Medical equipment
- Aerospace systems
- Embedded systems communication

2.2 Technical Characteristics

- **Multi-master**: Any node can transmit when bus is free
- **Message Priority**: Arbitration via identifier bits
- **Error Detection**: CRC, ACK, bit stuffing, frame check
- **Fault Confinement**: Automatic disconnection of faulty nodes
- **Speed**: Up to 1 Mbps at 40m cable length

2.3 CAN Frame Format

Field	Bits	Description
Start of Frame	1	Dominant bit (0)
Identifier	11/29	Message ID and priority
Control Field	6	DLC (Data Length Code)
Data Field	0-64	Payload data
CRC Field	15	Cyclic Redundancy Check
ACK Field	2	Acknowledgment slot
End of Frame	7	Recessive bits (1)

Table 2.1: CAN 2.0A Data Frame Structure

2.4 CAN Advantages for This Project

- Hardware-based error checking and correction

- Deterministic message delivery
- Suitable for real-time applications
- Robust in electrically noisy environments
- Standardized protocol with wide industry support

Chapter 3

System Overview

3.1 Global Architecture

The system implements a complete CAN communication chain from message generation to PC monitoring:

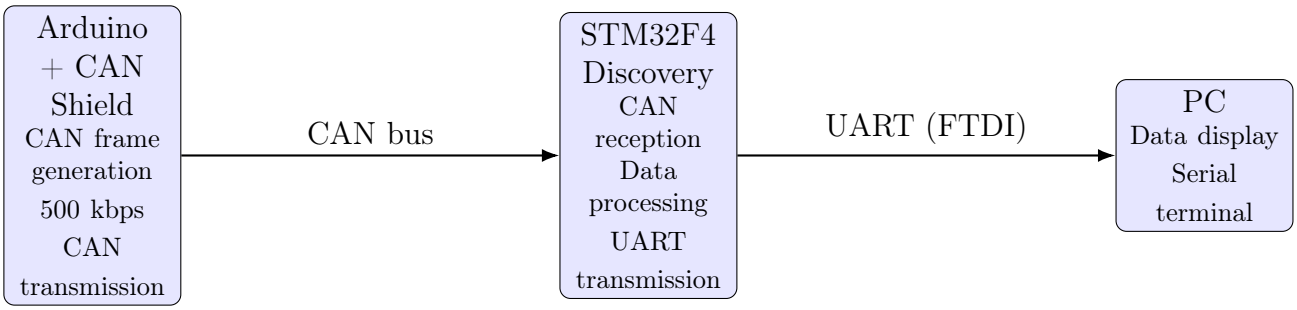

Figure 3.1: System Architecture Diagram

3.2 Communication Chain

The data flow follows this sequence:

1. Arduino generates CAN frames with test data
2. CAN bus transmits frames
3. STM32F4 receives frames via integrated CAN controller
4. Data is processed and formatted for serial transmission
5. UART transmits formatted data to PC via FTDI
6. PC terminal displays real-time CAN traffic

3.3 System Diagram

This diagram presents the overall embedded system architecture within a single system boundary. An Arduino UNO equipped with a CAN shield communicates with an STM32F407 microcontroller via a CAN bus. The STM32 processes the received data and forwards it to a PC through a USB–UART interface for monitoring and debugging.

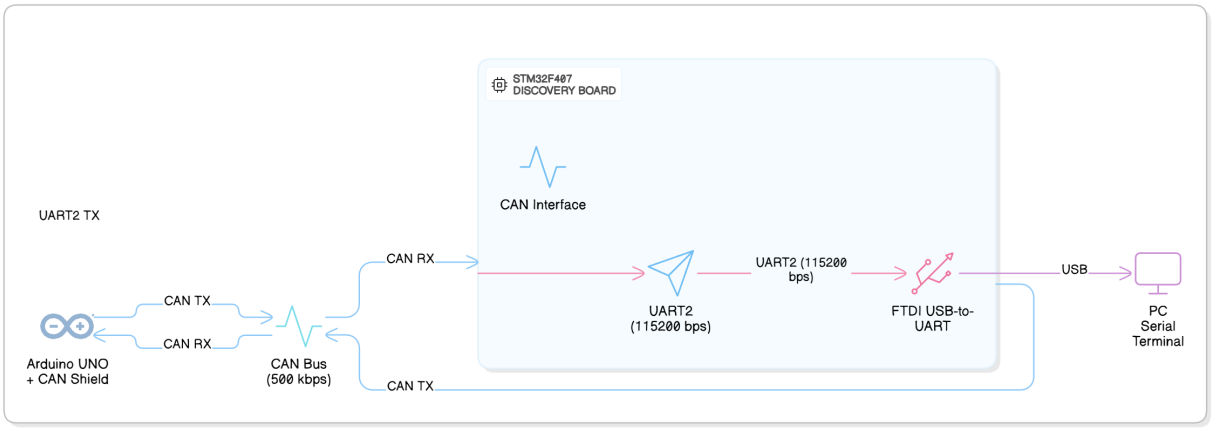

Figure 3.2: System Diagram

Chapter 4

Hardware Implementation

4.1 Equipment List

Component	Model	Purpose
Microcontroller	STM32F407VGT6	CAN reception and processing
Development Board	STM32F4 Discovery	Main processing unit
CAN Node	Arduino UNO + MCP2515	CAN frame generation
CAN Transceiver	MCP2551	CAN bus physical layer
USB-UART Converter	FT232RL	PC communication interface
CAN Termination	120 Ohms resistors	Bus impedance matching

Table 4.1: Equipment and Components

4.2 Clock Configuration (PLL)

Proper clock configuration is essential for accurate CAN timing:

Parameter	Value
PLL Source	HSI (16 MHz)
PLLM	8
PLLN	84
PLLP	2
SYSCLK Frequency	84 MHz
AHB Prescaler	1
APB1 Prescaler	2
APB1 Frequency	42 MHz
APB2 Prescaler	1
APB2 Frequency	84 MHz

Table 4.2: System Clock Configuration

Justification: APB1 at 42 MHz meets CAN peripheral requirements (max 45 MHz for CAN communication).

4.3 GPIO Configuration and Pinout

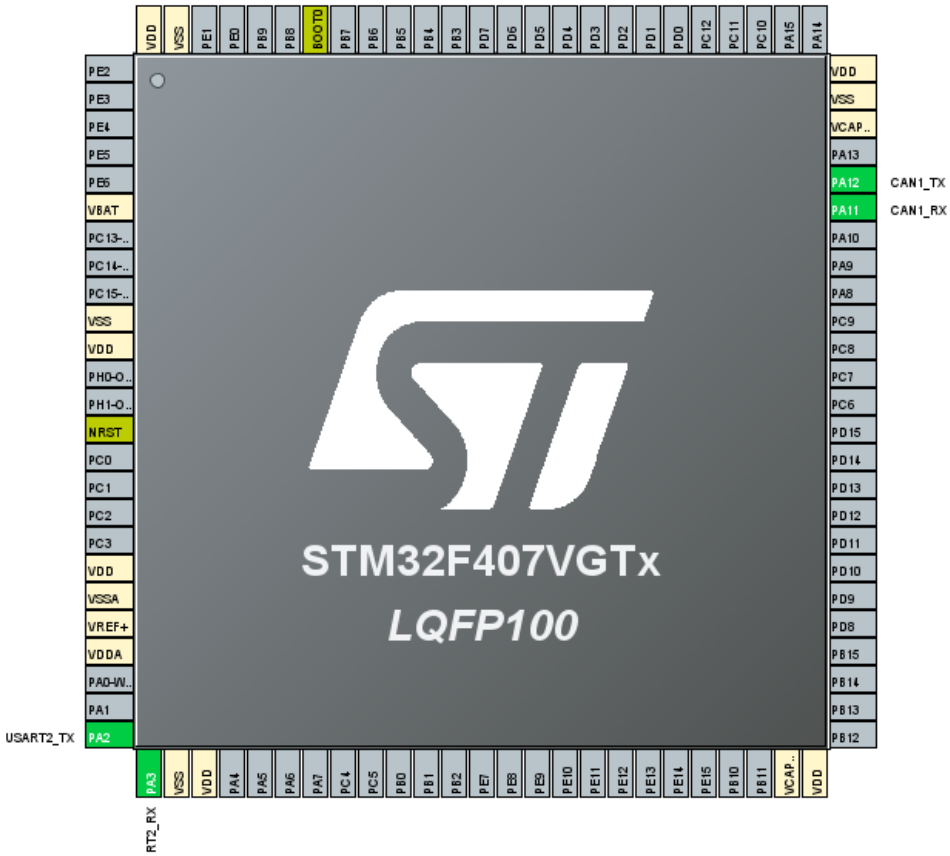

Figure 4.1: STM32CubeMX Pin Configuration

Pin	Signal	Function	Configuration
PA11	CAN1_RX	CAN Receive	AF9, Pull-up
PA12	CAN1_TX	CAN Transmit	AF9, Pull-up
PA2	USART2_TX	UART Transmit	AF7
PA3	USART2_RX	UART Receive	AF7

Table 4.3: GPIO Pin Assignment

4.4 CAN Configuration

4.4.1 Bit Timing Parameters

Parameter	Value
CAN Baud Rate	500 kbps
Prescaler	6
Time Segment 1 (BS1)	11 TQ
Time Segment 2 (BS2)	2 TQ
Synchronization Jump Width	1 TQ
Sample Point	87.5%
Time Quantum	142.857 ns
Bit Time	2 µs

Table 4.4: CAN Bit Timing Configuration

4.4.2 Baud Rate Calculation

$$\text{Baud Rate} = \frac{\text{APB1 Clock}}{\text{Prescaler} \times (\text{BS1} + \text{BS2} + 1)}$$

$$\text{Baud Rate} = \frac{42,000,000}{6 \times (11 + 2 + 1)} = \frac{42,000,000}{84} = 500,000 \text{ bps}$$

4.5 UART Configuration

Parameter	Value
UART Peripheral	USART2
Baud Rate	115200 bps
Word Length	8 bits
Parity	None
Stop Bits	1
Flow Control	None

Table 4.5: UART Communication Parameters

Chapter 5

Software Architecture

5.1 Program Structure

The firmware follows a modular architecture using STM32CubeMX HAL framework:

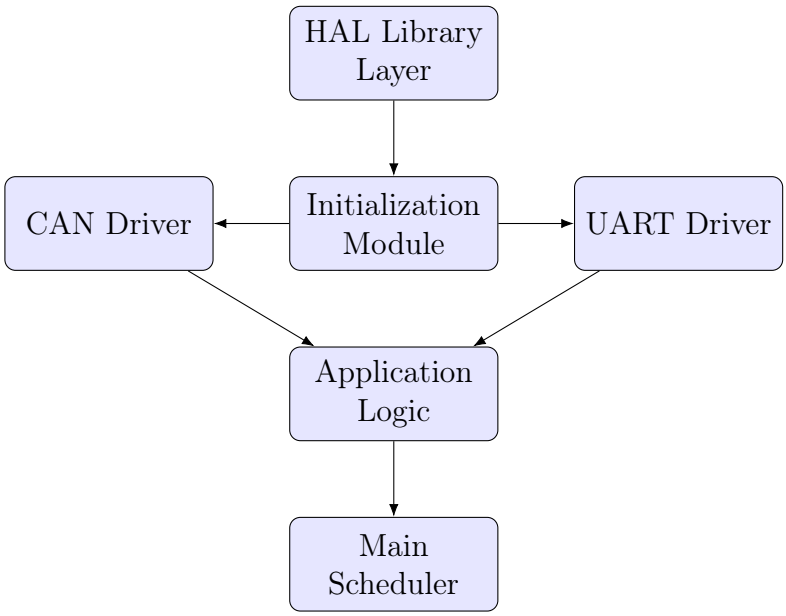

Figure 5.1: Software Architecture Layers

5.2 Key Registers Used

The configuration file ".ioc" generated by STMCubeMX sets up an STM32F407VGTx microcontroller for several tasks, primarily involving CAN and UART communication. The main peripherals configured are RCC (for clock setup), GPIO (for pin configuration), CAN1, USART2, and NVIC (for interrupt handling).

5.2.1 RCC (Reset and Clock Control)

The RCC peripheral is responsible for configuring the system clocks. A stable and correct clock configuration is fundamental for the operation of the microcontroller and its peripherals.

Key Registers and Configuration

- **RCC_CR (Clock Control Register)**:
  - **Role**: Controls the main oscillators (HSI, HSE, PLL)
  - **Configuration**:

Table 5.1: RCC Clock Configuration Summary

Parameter	Value
System Clock (SYSCLK)	84 MHz
PLL Source	HSI (16 MHz)
AHB Clock (HCLK)	84 MHz
APB1 Clock (PCLK1)	42 MHz
APB2 Clock (PCLK2)	84 MHz

    - HSION: Bit 0 is set to 1 to enable the High-Speed Internal oscillator (HSI)
    - PLLON: Bit 24 is set to 1 to enable the main PLL

- **RCC_PLLCFGR (PLL Configuration Register)**:
  - **Role**: Configures the PLL multiplication and division factors
  - **Configuration**:
    - PLLSRC: Bit 22 = 0 (select HSI as PLL input)
    - PLLM: Bits 5:0 = 8 (16 MHz / 8 = 2 MHz)
    - PLLN: Bits 14:6 = 84 (2 MHz * 84 = 168 MHz)
    - PLLP: Bits 17:16 = 00 (168 MHz / 2 = 84 MHz)
- **RCC_CFGR (Clock Configuration Register)**:
  - **Role**: Selects system clock source and configures bus prescalers
  - **Configuration**:
    - SW: Bits 1:0 = 10 (select PLL as system clock)
    - HPRE: Bits 7:4 = 0xxx (HCLK = 84 MHz)
    - PPRE1: Bits 12:10 = 100 (PCLK1 = 42 MHz)
    - PPRE2: Bits 15:13 = 0xx (PCLK2 = 84 MHz)

5.2.2 GPIO (General Purpose Input/Output)

GPIO pins are configured to connect the microcontroller's internal peripherals (CAN1, USART2) to the outside world.

Table 5.2: GPIO Pin Configuration

Pin	Function	Alternate Function
PA11	CAN1_RX	AF9
PA12	CAN1_TX	AF9
PA2	USART2_TX	AF7
PA3	USART2_RX	AF7

Key Registers and Configuration

- **GPIOA_MODER (GPIO Port Mode Register)**:
  - **Role**: Configures I/O direction mode

  - MODER2, MODER3, MODER11, MODER12 = 10 (Alternate Function mode)

- **GPIOA_AFRL & GPIOA_AFRH (Alternate Function Registers)**:
  - **Role**: Selects specific alternate function for each pin
  - AFRL2 = 0111 (AF7 - USART2_TX for PA2)
  - AFRL3 = 0111 (AF7 - USART2_RX for PA3)
  - AFRH3 = 1001 (AF9 - CAN1_RX for PA11)
  - AFRH4 = 1001 (AF9 - CAN1_TX for PA12)

5.2.3 CAN1 (Controller Area Network)

The CAN peripheral is set up for communication on a CAN bus at a speed of 500 kbit/s.

Table 5.3: CAN1 Configuration Summary

Parameter	**Value**
Baud Rate	500 kbit/s
APB1 Clock	42 MHz
Prescaler	6
BS1	11 tq
BS2	2 tq
Total Bit Time	14 tq

Key Registers and Configuration

- **CAN_MCR (Master Control Register)**:
  - **Role**: Controls operating mode
  - INRQ: Bit 0 = 1 (initialization mode)
  - NART: Bit 4 = 0 (enable automatic retransmission)
- **CAN_BTR (Bit Timing Register)**:
  - **Role**: Defines bit timing for baud rate
  - BRP: Bits 9:0 = 5 (Prescaler = 6)
  - TS1: Bits 19:16 = 10 (BS1 = 11 tq)
  - TS2: Bits 22:20 = 1 (BS2 = 2 tq)
  - Baud Rate = 42 MHz / (6 × 14) = 500,000 bit/s

5.2.4 USART2 (Universal Synchronous/Asynchronous Receiver/-Transmitter)

USART2 is configured for standard asynchronous serial communication.

Key Registers and Configuration

- **USART_CR1 (Control Register 1)**:
  - **Role**: Enables USART and configures basic parameters
  - UE: Bit 13 = 1 (enable USART)
  - TE: Bit 3 = 1 (enable transmitter)
  - RE: Bit 2 = 1 (enable receiver)
- **USART_BRR (Baud Rate Register)**:
  - **Role**: Configures baud rate
  - For 115200 baud with 42 MHz clock:
    - USARTDIV = 42,000,000 / (16 × 115200) = 22.786
    - DIV_Mantissa = 22 (0x16)
    - DIV_Fraction = 13 (0xD)
    - BRR = 0x16D

5.2.5 NVIC (Nested Vectored Interrupt Controller)

The NVIC is configured to enable and prioritize system exceptions.

Key Registers and Configuration

- **NVIC_IPR (Interrupt Priority Registers)**:
  - **Role**: Sets priority for each interrupt/exception
  - SysTick_IRQn priority = 15 (lowest)
  - Fault exceptions priority = 0 (highest)
- **SCB→SHPR (System Handler Priority Registers)**:
  - **Role**: Sets priorities for system exceptions

This detailed explanation covers the register-level configuration derived from the DS.ioc file.

5.3 Program Flowchart

Figure **??** illustrates the global execution flow of the embedded program, including system initialization, the main idle loop, and the CAN receive interrupt routine.

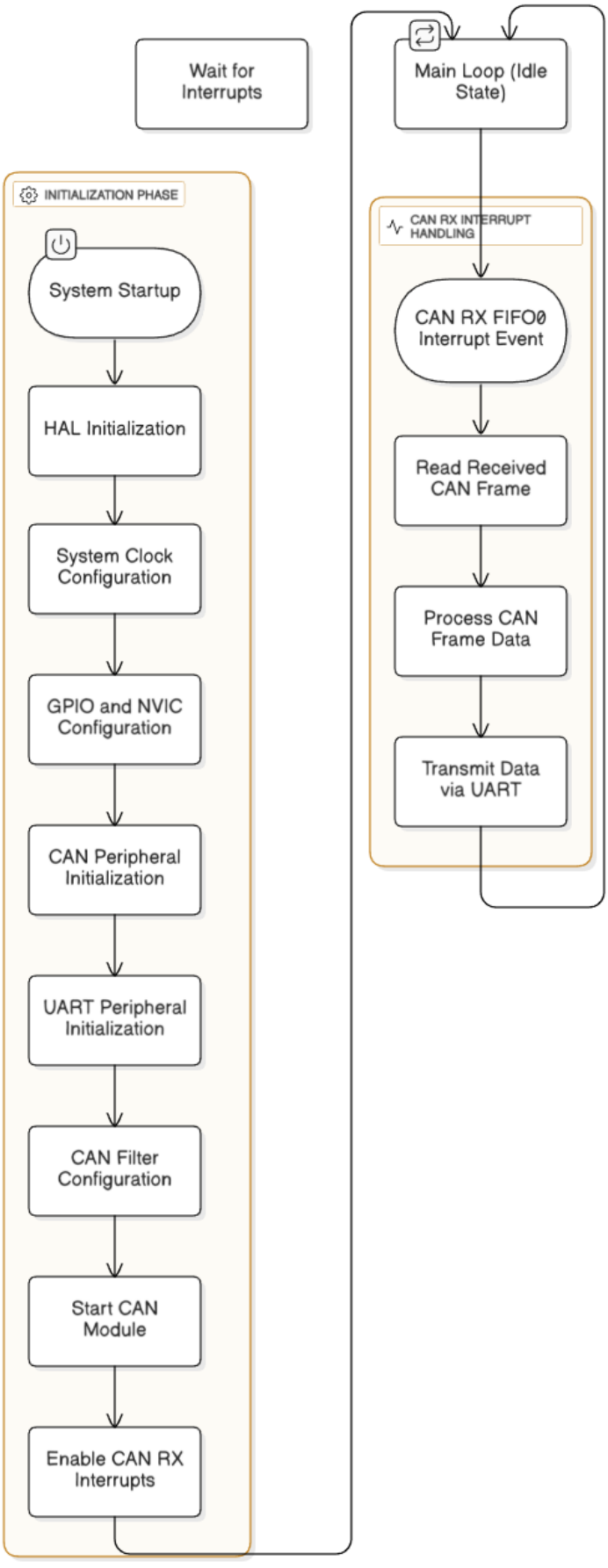

Figure 5.2: Program execution flow

5.4 CAN FIFO0 Reception and Interrupt Management

The system uses interrupt-driven CAN reception for efficient operation:

1. CAN frames received via FIFO0
2. Hardware triggers RX interrupt on message pending
3. Interrupt service routine reads frame data
4. Data processed and prepared for UART transmission
5. Normal program execution resumes

5.5 UART Transmission to PC

Received CAN frames are formatted and transmitted to PC via UART:

- ASCII formatting for human readability
- Includes CAN ID, DLC, and data bytes
- Timestamp and frame counter for analysis
- Error reporting for communication issues

Chapter 6

Embedded Software Implementation

This chapter presents the implementation of the embedded firmware developed for CAN communication and real-time monitoring via UART. The system is based on the STM32F407 microcontroller and uses the HAL library.

6.1 CAN Peripheral Configuration

The CAN peripheral is configured in normal operating mode with a bit timing suitable for reliable communication. GPIO configuration, peripheral clock enabling, and NVIC interrupt setup are handled in the `HAL_CAN_MspInit()` function generated by STM32CubeMX.

```c
#include "main.h"
#include <string.h>
#include <stdio.h>

CAN_HandleTypeDef hcan1;
UART_HandleTypeDef huart2;

/* Global variables for CAN reception */
CAN_RxHeaderTypeDef RxHeader;
uint8_t RxData[8];
char uart_buffer[128];

/**
  * @brief CAN1 Initialization Function
  * @retval None
  */
void MX_CAN1_Init(void)
{
    hcan1.Instance = CAN1;
    hcan1.Init.Prescaler = 6;
    hcan1.Init.Mode = CAN_MODE_NORMAL;
    hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
    hcan1.Init.TimeSeg1 = CAN_BS1_11TQ;
    hcan1.Init.TimeSeg2 = CAN_BS2_2TQ;
    hcan1.Init.TimeTriggeredMode = DISABLE;
    hcan1.Init.AutoBusOff = DISABLE;
    hcan1.Init.AutoWakeUp = DISABLE;
    hcan1.Init.AutoRetransmission = ENABLE;
    hcan1.Init.ReceiveFifoLocked = DISABLE;
    hcan1.Init.TransmitFifoPriority = DISABLE;

    if (HAL_CAN_Init(&hcan1) != HAL_OK)
    {
        Error_Handler();
    }
}
```

Listing 6.1: CAN Initialization and Configuration

6.2 CAN Filter Configuration

A CAN filter is configured to accept all standard and extended identifiers. Since only CAN1 is used in this application, the slave filter bank parameter has no functional impact.

```
/**
  * @brief CAN Filter Configuration
  * Accept all CAN identifiers
  */
void CAN_Filter_Config(void)
{
    CAN_FilterTypeDef sFilterConfig;

    sFilterConfig.FilterBank = 0;
    sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;
    sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;
    sFilterConfig.FilterIdHigh = 0x0000;
    sFilterConfig.FilterIdLow = 0x0000;
    sFilterConfig.FilterMaskIdHigh = 0x0000;
    sFilterConfig.FilterMaskIdLow = 0x0000;
    sFilterConfig.FilterFIFOAssignment = CAN_FILTER_FIFO0;
    sFilterConfig.FilterActivation = ENABLE;
    sFilterConfig.SlaveStartFilterBank = 14;

    if (HAL_CAN_ConfigFilter(&hcan1, &sFilterConfig) != HAL_OK)
    {
        Error_Handler();
    }
}
```

Listing 6.2: CAN Filter Configuration

6.3 UART Configuration

USART2 is used for serial communication with a PC. It is mapped to pins PA2 (TX) and PA3 (RX) and connected internally to the ST-LINK Virtual COM Port, enabling real-time visualization of received CAN frames. GPIO and clock configurations are handled in `HAL_UART_MspInit()`.

```
/**
  * @brief USART2 Initialization Function
  * @retval None
  */
void MX_USART2_UART_Init(void)
{
    huart2.Instance = USART2;
    huart2.Init.BaudRate = 115200;
    huart2.Init.WordLength = UART_WORDLENGTH_8B;
    huart2.Init.StopBits = UART_STOPBITS_1;
    huart2.Init.Parity = UART_PARITY_NONE;
    huart2.Init.Mode = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart2.Init.OverSampling = UART_OVERSAMPLING_16;

    if (HAL_UART_Init(&huart2) != HAL_OK)
    {
        Error_Handler();
```

```
    }
}
```

Listing 6.3: USART2 Initialization

6.4 CAN Reception Callback and UART Output

CAN reception is handled using an interrupt-driven approach. When a message is received in FIFO0, the HAL automatically invokes a callback function, ensuring low-latency processing.

```
/**
  * @brief CAN RX FIFO0 message pending callback
  */
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{
    if (hcan->Instance == CAN1)
    {
        if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData)
    == HAL_OK)
        {
            Process_CAN_Frame();
        }
    }
}

/**
  * @brief Process received CAN frame and transmit via UART
  */
void Process_CAN_Frame(void)
{
    uint32_t id;
    uint8_t i;
    int len;

    if (RxHeader.IDE == CAN_ID_STD)
    {
        id = RxHeader.StdId;
    }
    else
    {
        id = RxHeader.ExtId;
    }

    len = snprintf(uart_buffer, sizeof(uart_buffer),
                   "CAN Frame -> ID: 0x%03lX, DLC: %lu, Data:",
                   id, RxHeader.DLC);

    for (i = 0; i < RxHeader.DLC && len < sizeof(uart_buffer); i++)
    {
        len += snprintf(&uart_buffer[len],
                        sizeof(uart_buffer) - len,
                        " %02X", RxData[i]);
    }

    len += snprintf(&uart_buffer[len],
                    sizeof(uart_buffer) - len, "\r\n");

    HAL_UART_Transmit(&huart2,
```

```
                        (uint8_t *)uart_buffer,
                        len, 100);
}
```

Listing 6.4: CAN Reception Callback and Data Processing

6.5 CAN Communication Startup

This function initializes the CAN filters, starts the CAN peripheral, and enables reception interrupts. A startup message is transmitted via UART to confirm system readiness.

```
/**
  * @brief Start CAN communication and enable interrupts
  */
void Start_CAN_Communication(void)
{
    CAN_Filter_Config();

    if (HAL_CAN_Start(&hcan1) != HAL_OK)
    {
        Error_Handler();
    }

    if (HAL_CAN_ActivateNotification(&hcan1,
        CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK)
    {
        Error_Handler();
    }

    HAL_UART_Transmit(&huart2,
        (uint8_t *)
        "CAN Communication Started - Ready to receive frames...\r\n",
        59, 100);
}
```

Listing 6.5: CAN Startup Procedure

Appendix A

Complete Source Code

```
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  *
  * Project : STM32F4 CAN sniffer
  * Function:
  *   - Receive CAN frames on CAN1 (from Arduino + CAN shield)
  *   - Forward frames as ASCII text to PC via USART2 (FTDI)
  *
  * Toolchain:
  *   - STM32CubeMX for code generation (CAN, USART, GPIO)
  *   - Keil MDK-ARM for compilation and debugging
  *
  ******************************************************************************
  */

#include "main.h"
#include "can.h"
#include "usart.h"
#include "gpio.h"
#include <stdio.h>
#include <string.h>

/*
 ------------------------------------------------------------------------------
  */
/*                        Global variables
          */


/* CAN reception header and data buffer (max 8 bytes in classic CAN)
         */
CAN_RxHeaderTypeDef RxHeader;
uint8_t RxData[8];

/* Small buffer used to format text sent over UART
          */
char uart_buffer[128];

/*
 ------------------------------------------------------------------------------
  */
/*                        Function prototypes
          */
```

```
38  /*
     ----------------------------------------------------------------------
      */
39
40  void SystemClock_Config(void);
41  static void CAN_Filter_Config(void);
42  void Start_CAN_Communication(void);
43  volatile uint8_t can_msg_ready = 0; //nouveau
44
45
46  /*
     ----------------------------------------------------------------------
      */
47  /*                      Retarget printf to USART2
          */
48  /*
     ----------------------------------------------------------------------
      */
49  /**
50    * @brief  Redirects printf() to USART2.
51    *         This allows using printf() for debugging through the FTDI
     adapter.
52    * @param  file: not used
53    * @param  ptr : pointer to data
54    * @param  len : length of data
55    * @retval number of bytes transmitted
56    */
57  int _write(int file, char *ptr, int len)
58  {
59      HAL_UART_Transmit(&huart2, (uint8_t *)ptr, len, HAL_MAX_DELAY);
60      return len;
61  }
62
63  /*
     ----------------------------------------------------------------------
      */
64  /*                                  MAIN
          */
65  /*
     ----------------------------------------------------------------------
      */
66
67  int main(void)
68  {
69      /* 1) Initialize the HAL library:
70            - Configure Flash prefetch, instruction cache, data cache
71            - Configure SysTick timer (1 ms)
72       */
73      HAL_Init();
74
75      /* 2) Configure the system clock:
76            - HSI (16 MHz) used as PLL source
77            - PLL ? SYSCLK = 84 MHz
78            - APB1 = 42 MHz (CAN, USART2)
79            - APB2 = 84 MHz
80       */
81      SystemClock_Config();
82
```

```
83      /* 3) Initialize all configured peripherals (generated by CubeMX)
            */
84      MX_GPIO_Init();        /* GPIO (optional: LEDs, buttons, etc.)
            */
85      MX_CAN1_Init();
86          MX_USART2_UART_Init();
87      /* CAN1 (parameters from CubeMX)                */
88       /* USART2 (115200 8N1 for FTDI)                 */
89
90      /* 4) Demarrer la communication CAN avec configuration compl te:
91            - Configurer les filtres (accepter tous les ID)
92            - D marrer le module CAN
93            - Activer les interruptions FIFO0
94       */
95      Start_CAN_Communication();
96
97      /* 5) Boucle principale :
98            - Toute la r ception CAN est g r e par le callback d'
    interruption
99            - Ici, vous pouvez ajouter d'autres t ches de fond si
    n cessaire
100           - La consommation CPU est tr s faible (mode attente)
101       */
102     while (1)
103     {
104         /* Boucle d'inactivit      faible consommation CPU, tout le
    travail */
105         /* est effectu  dans l'IRQ. Vous pouvez placer ici :
             */
106         /* - la gestion de la faible consommation d' nergie
             */
107         /* - d'autres t ches de fond (watchdog, tests LED, etc.)
            */
108     }
109 }
110
111 /*
     ------------------------------------------------------------------------
      */
112 /*                     CAN filter configuration
             */
113 /*
     ------------------------------------------------------------------------
      */
114 /**
115   * @brief  Configurer les filtres CAN1 pour accepter tous les ID
    vers FIFO0.
116   *         Configuration de type "sniffer" : pas de filtrage.
117   *
118   *         Chaque trame CAN re ue sera traitee dans le callback de
    FIFO0.
119   * @param  None
120   * @retval None
121   */
122 static void CAN_Filter_Config(void)
123 {
124     CAN_FilterTypeDef sFilterConfig;
125
```

```
126         /* Banque de filtre 0 (premiere banque disponible)
                */
127         sFilterConfig.FilterBank = 0;
128 
129         /* Mode ID-MASK : compare l'ID re u avec le masque sp cifi
                 */
130         sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;
131 
132         /*   chelle   32 bits (ID + masque dans un seul registre 32 bits)
                */
133         sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;
134 
135         /* Accepter tous les ID standards et   tendus   :
                */
136         /* ID = 0x00000000, MASK = 0x00000000 (accepte tout)
               */
137         sFilterConfig.FilterIdHigh      = 0x0000;   /* Bits 31-16 de l'ID
               */
138         sFilterConfig.FilterIdLow       = 0x0000;   /* Bits 15-0 de l'ID
               */
139         sFilterConfig.FilterMaskIdHigh  = 0x0000;   /* Bits 31-16 du
    masque       */
140         sFilterConfig.FilterMaskIdLow   = 0x0000;   /* Bits 15-0 du
    masque        */
141 
142         /* Diriger les messages filtr s vers FIFO 0
                */
143         sFilterConfig.FilterFIFOAssignment = CAN_FILTER_FIFO0;
144 
145         /* Activer ce filtre
               */
146         sFilterConfig.FilterActivation = ENABLE;
147 
148         /* Pour une instance CAN unique sur STM32F4, c'est typiquement 14
                */
149         /* C'est le point o   les filtres "esclaves" commencent
                */
150         sFilterConfig.SlaveStartFilterBank = 14;
151 
152         if (HAL_CAN_ConfigFilter(&hcan1, &sFilterConfig) != HAL_OK)
153         {
154             Error_Handler();
155         }
156     }
157 
158     /*
      ----------------------------------------------------------------------
       */
159     /*                     CAN RX FIFO0 interrupt callback
             */
160     /*
      ----------------------------------------------------------------------
       */
161     /**
162       * @brief  Traiter une trame CAN recue et l'envoyer via UART
163       *
164       *         Cette fonction formate la trame CAN avec :
165       *         - L'ID (11 bits pour standard, 29 bits pour etendu)
166       *         - Le DLC (nombre d'octets de donnees)
```

```
167     *          - Les octets de donn es en hexadecimal
168     *
169     *          Format de sortie : "CAN Frame -> ID: 0xXXX, DLC: X, Data:
        XX XX XX ..."
170     * @param  None
171     * @retval None
172     */
173   void Process_CAN_Frame(void)
174   {
175       uint32_t id;
176       uint8_t i;
177
178       /* Determiner le type de trame (Standard ou Etendue)
              */
179       if (RxHeader.IDE == CAN_ID_STD)
180       {
181           id = RxHeader.StdId;  /* ID standard 11 bits
               */
182       }
183       else
184       {
185           id = RxHeader.ExtId;  /* ID   tendu   29 bits
                */
186       }
187
188       /* Formater la cha ne de sortie
              */
189       int len = snprintf(uart_buffer, sizeof(uart_buffer),
190                          "CAN Frame -> ID: 0x%03lX, DLC: %lu, Data:", id
     , RxHeader.DLC);
191
192       /* Ajouter les octets de donn es
              */
193       for (i = 0; i < RxHeader.DLC; i++)
194       {
195           len += snprintf(&uart_buffer[len], sizeof(uart_buffer) - len,
196                           " %02X", RxData[i]);
197       }
198
199       /* Ajouter le saut de ligne
             */
200       len += snprintf(&uart_buffer[len], sizeof(uart_buffer) - len, "\r
     \n");
201
202       /* Transmettre via UART avec timeout de 100 ms
             */
203       HAL_UART_Transmit(&huart2, (uint8_t*)uart_buffer, len, 100);
204   }
205
206   /**
207     * @brief USART2 Initialization Function
208     * @param None
209     * @retval None
210     */
211
212
213   /**
214     * @brief  Callback d'interruption FIFO0 du module CAN
215     *
```

```
216      *          Cette fonction est appel e automatiquement lorsqu'une
     trame CAN
217      *          est re ue et plac e dans FIFO0. Elle est invoqu e par
     l'IRQ
218      *          CAN1_RX0_IRQHandler via la couche HAL.
219      *
220      *          Proc dure :
221      *          1. V rifier que l'interruption vient bien de CAN1
222      *          2. R cup rer la trame depuis FIFO0
223      *          3. Appeler la fonction de traitement
224      * @param  hcan: pointeur sur la structure de gestion du CAN
225      * @retval None
226      */
227 //Nouvelle version
228   void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
229 {
230     if (hcan->Instance == CAN1)
231     {
232         if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData)
     == HAL_OK)
233         {
234             can_msg_ready = 1;
235         }
236     }
237 }
238
239   /**
240     * @brief  D marrer la communication CAN et activer les
     interruptions
241     *
242     *          Cette fonction effectue les  tapes  suivantes :
243     *          1. Configurer les filtres CAN (pour le sniffing de tous
     les ID)
244     *          2. D marrer le module CAN (passer en mode Normal/Active)
245     *          3. Activer l'interruption de FIFO0
246     *          4. Afficher un message de confirmation sur UART
247     *
248     *          Apres cet appel, toutes les trames recues seront traitees
249     *          dans le callback HAL_CAN_RxFifo0MsgPendingCallback().
250     * @param  None
251     * @retval None
252     */
253   void Start_CAN_Communication(void)
254   {
255       /* Configurer les filtres CAN
              */
256       CAN_Filter_Config();
257
258       /* Demarrer le module CAN
              */
259       if (HAL_CAN_Start(&hcan1) != HAL_OK)
260       {
261           Error_Handler();
262       }
263
264       /* Activer l'interruption de FIFO0 (message en attente)
              */
265       if (HAL_CAN_ActivateNotification(&hcan1,
     CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK)
```

```
266         {
267             Error_Handler();
268         }
269 
270         /* Informer l'utilisateur que la communication est prete
                */
271         printf("CAN Communication Started - Ready to receive frames...\r\
    n");
272     }
273 
274     /*
      ------------------------------------------------------------------------
       */
275     /*                          System Clock Configuration
                */
276     /*
      ------------------------------------------------------------------------
       */
277     /**
278       * @brief  System Clock Configuration
279       *         HSI (16 MHz) ? PLL ? SYSCLK = 84 MHz
280       *         AHB  = 84 MHz
281       *         APB1 = 42 MHz  (CAN, USART2)
282       *         APB2 = 84 MHz
283       */
284     void SystemClock_Config(void)
285     {
286         RCC_OscInitTypeDef RCC_OscInitStruct = {0};
287         RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
288 
289         /* Enable Power control clock
                */
290         __HAL_RCC_PWR_CLK_ENABLE();
291         /* Voltage scaling
                */
292         __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE2);
293 
294         /* Configure main internal oscillator (HSI) and PLL
                */
295         RCC_OscInitStruct.OscillatorType        = RCC_OSCILLATORTYPE_HSI;
296         RCC_OscInitStruct.HSIState              = RCC_HSI_ON;
297         RCC_OscInitStruct.HSICalibrationValue   =
    RCC_HSICALIBRATION_DEFAULT;
298         RCC_OscInitStruct.PLL.PLLState          = RCC_PLL_ON;
299         RCC_OscInitStruct.PLL.PLLSource         = RCC_PLLSOURCE_HSI;
300         RCC_OscInitStruct.PLL.PLLM              = 8;
301         RCC_OscInitStruct.PLL.PLLN              = 84;
302         RCC_OscInitStruct.PLL.PLLP              = RCC_PLLP_DIV2;
303         RCC_OscInitStruct.PLL.PLLQ              = 4;
304 
305         if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
306         {
307             Error_Handler();
308         }
309 
310         /* Select PLL as system clock source and configure HCLK, PCLK1
    and PCLK2 */
311         RCC_ClkInitStruct.ClockType      = RCC_CLOCKTYPE_HCLK |
    RCC_CLOCKTYPE_SYSCLK |
```

```
312                                         RCC_CLOCKTYPE_PCLK1 |
    RCC_CLOCKTYPE_PCLK2;
313       RCC_ClkInitStruct.SYSCLKSource   = RCC_SYSCLKSOURCE_PLLCLK;
314       RCC_ClkInitStruct.AHBCLKDivider  = RCC_SYSCLK_DIV1;      /* HCLK
     = 84 MHz   */
315       RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;        /* PCLK1
     = 42 MHz   */
316       RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;        /* PCLK2
     = 84 MHz   */
317
318       if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) !=
    HAL_OK)
319       {
320           Error_Handler();
321       }
322   }
323
324   /*
     ----------------------------------------------------------------------
      */
325   /*                                 Error Handler
           */
326   /*
     ----------------------------------------------------------------------
      */
327   /**
328     * @brief  Error handler: infinite loop with interrupts disabled.
329     *         Can be improved by toggling an LED or sending debug info.
330     */
331   void Error_Handler(void)
332   {
333       __disable_irq();
334       while (1)
335       {
336           /* Stay here to help debugging
            */
337       }
338   }
```

Listing A.1: Complete source code

Appendix B

Additional Documentation

B.1 Datasheets and References

- STM32F407xx Reference Manual (RM0090)
- STM32F4 HAL Library Documentation
- MCP2515 CAN Controller Datasheet
- MCP2551 CAN Transceiver Datasheet
- CAN 2.0 Specification